Social Network Analysis of the Canadian Disability Rights Community

Mapping Collaboration Networks and Key Actors in Disability Rights Advocacy

Dr. Derrick L. Cogburn

2026-03-28

Table of contents

1 Executive Summary **1**

2 Data Cleaning and Pipeline **1**
2.1 Source Data . . . 1
2.2 Survey Structure and Key Fields . . . 2
2.3 Data Cleaning Steps . . . 2
2.4 Output Files . . . 3

3 Setup **3**
3.1 Install Required Packages . . . 3
3.2 Load Libraries . . . 3
3.3 Import Data . . . 4
3.4 Colour Palettes . . . 4

4 Two-Mode Bipartite Network: Respondents × Issue Areas **6**

5 One-Mode Projected Network: Respondent Collaboration **8**
5.1 Network Structure Metrics . . . 11

6 Centrality Analysis **12**

7 Community Detection **14**
7.1 Community Composition . . . 16

8 Degree Distributions and Collaboration Patterns **17**

9 Discussion and Implications **19**
9.1 Fragmentation as a structural feature 19
9.2 The role of national organisations 19
9.3 Bridging actors 19
9.4 Limitations 20

1 Executive Summary

This report presents a Social Network Analysis (SNA) of collaboration patterns within the Canadian disability rights community, based on a bilingual survey (English/French). The survey, called "Enquête d'analyse des Réseaux Sociaux / Social Network Analysis Survey" enables an analysis and mapping of who collaborates with whom, identifies the most central actors and organizations, detects community clusters, and assesses overall network structure.

2 Data Cleaning and Pipeline

2.1 Source Data

The raw data was collected via a bilingual (English/French) Microsoft Forms survey distributed to members of the Canadian disability rights community. The survey instrument contained 51 columns across a single worksheet, with parallel English and French versions of each question. A total of 111 responses were recorded. The raw data file was de-identified prior to analysis.

2.2 Survey Structure and Key Fields

The survey captured several categories of information relevant to social network analysis. Respondent attributes included the respondent's connection to the disability community (e.g., person with a disability, family member/caregiver, advocate, researcher, health professional) and the disability rights issues in which they are most actively involved (free-text responses covering topics such as employment, children/youth, housing, poverty, and accessibility). Collaboration data included the names of up to three organizations or individuals with whom the respondent has partnered in the past five years (free-text), the frequency of collaboration with each named partner (regularly, occasionally, or seldom), and whether the respondent's named partners are linked to each other.

2.3 Data Cleaning Steps

The cleaning pipeline proceeded through several stages. First, the metadata row generated by Microsoft Forms was removed, and the language preference field was used to route each respondent to the appropriate English or French column for each question. Second, the bilingual fields were consolidated, with the French-language response used when the respondent selected French and the English-language response used otherwise. This produced a unified respondent table with consistent column names regardless of survey language.

Third, connection types were standardized into nine categories: Person with Disability, Advocate/CSO, Family/Caregiver, Researcher, Health, Education, Leisure, Government/Policy, and Other. Respondents whose connection type text matched multiple categories were assigned all applicable categories (semicolon-separated), with the first match designated as the primary connection type.

Fourth, disability rights issue areas were categorized from the free-text responses into 12 standardized topics: Children/Youth, Employment, Accessibility/Inclusion, Poverty/Income, Rights/Policy, Education, Housing, Health, Indigenous, Transportation, Technology, and Other. As with connection types, respondents active in multiple areas received all applicable categories.

Fifth, the free-text partner names were parsed into individual organization or person names. The parser handled several formats including comma-separated lists, semicolon-separated lists, numbered lists, and narrative descriptions. Each respondent contributed up to three partner names. Names were then normalized by converting to lowercase, collapsing whitespace, and removing punctuation. A prefix-matching algorithm was applied to merge near-duplicate entries (e.g., variant spellings or abbreviations of the same organization). This best-effort automated cleaning reduced the number of unique partner names from 175 to 166.

Sixth, collaboration frequency values were harmonized across languages, mapping both English terms (Regularly, Occasionally, Seldom) and their French equivalents (Régulièrement, Occasionnellement, Rarement) to a common three-level scale plus an Unknown category for non-standard responses.

2.4 Output Files

The pipeline produced two analysis-ready CSV files. The node table (`respondents_final.csv`) contains one row per respondent with fields for respondent ID, language, raw and categorized connection type, raw and categorized issue areas, and primary connection and issue assignments. The edge table (`edges_final.csv`) contains one row per respondent-partner tie with fields for respondent ID, raw partner name, normalized partner name, cleaned canonical partner name, raw frequency, and harmonized frequency.

3 Setup

3.1 Install Required Packages

Run this chunk once to install all dependencies. After installation, you can set `eval: false` or simply skip it on future renders.

```
import subprocess, sys
for pkg in ["networkx", "pandas", "matplotlib", "seaborn",
            "python-louvain", "tabulate"]:
    subprocess.check_call([sys.executable, "-m", "pip",
                           "install", pkg, "-q"])
```

3.2 Load Libraries

```
import pandas as pd
import numpy as np
import networkx as nx
import community.community_louvain as community_louvain
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.lines import Line2D
from collections import Counter, defaultdict
from tabulate import tabulate
from IPython.display import Markdown

plt.rcParams.update({
    "figure.dpi": 150,
    "savefig.dpi": 150,
    "font.family": "sans-serif",
    "font.size": 9,
})
np.random.seed(42)
```

3.3 Import Data

The analysis uses two pre-cleaned CSV files produced by Dr. Cogburn in a data-preparation pipeline. These files, `respondents_final.csv` and `edges_final.csv` are located in the data

folder, which is located in the same directory as this `.qmd` file. Paths to the data should be updated to data/filename.

```
#   Node table: one row per survey respondent
respondents = pd.read_csv("data/respondents_final.csv")
respondents = respondents[respondents["resp_id"] != "RMicrosoft Forms "]

#   Edge table: one row per respondent → partner tie
edges = pd.read_csv("data/edges_final.csv")
edges = edges[
    edges["partner_clean"].notna()
    & (edges["partner_clean"] != "nan")
    & (edges["partner_clean"] != "None")
]
```

```
print(f"Respondents: {len(respondents)}")
print(f"Respondents who named partners: {edges['respondent'].nunique()}")
print(f"Unique organisations named: {edges['partner_clean'].nunique()}")
print(f"Total collaboration ties: {len(edges)}")
```

```
Respondents: 110
Respondents who named partners: 57
Unique organisations named: 166
Total collaboration ties: 182
```

3.4 Colour Palettes

```
# Connection-type palette (respondent node colouring)
conn_palette = {
    "Person with Disability": "#E63946",
    "Advocate/CSO":           "#457B9D",
    "Family/Caregiver":       "#2A9D8F",
    "Researcher":             "#6A4C93",
    "Health":                 "#BC4749",
    "Education":              "#F4A261",
    "Leisure":                "#8AC926",
    "Government/Policy":      "#264653",
    "Other":                  "#AAAAAA",
    "Unknown":                "#BBBBBB",
}
```

```
# Issue-area palette (bipartite squares)
issue_palette = {
    "Children/Youth":          "#E63946",
    "Employment":              "#457B9D",
    "Accessibility/Inclusion": "#2A9D8F",
    "Poverty/Income":          "#E9C46A",
    "Rights/Policy":           "#264653",
    "Education":               "#F4A261",
    "Housing":                 "#606C38",
    "Health":                  "#BC4749",
    "Indigenous":              "#6A4C93",
    "Transportation":          "#1982C4",
    "Technology":              "#8AC926",
    "Other":                   "#AAAAAA",
}

# Collaboration-frequency palette
freq_palette = {
    "Regularly":    "#E63946",
    "Occasionally": "#E9C46A",
    "Seldom":       "#457B9D",
    "Unknown":      "#BBBBBB",
}
```

4 Two-Mode Bipartite Network: Respondents × Issue Areas

This network maps respondents (circles) to the disability rights issue areas they are most actively involved in (squares). Respondents are coloured by their connection type to the disability community; issue-area nodes are sized by the number of affiliated respondents.

```
# Build bipartite graph: respondents  issue areas
B = nx.Graph()

for _, row in respondents.iterrows():
    issue_cat = row["issue_category"]
    if not isinstance(issue_cat, str) or issue_cat in ("Unknown", "nan"):
        continue
    B.add_node(row["resp_id"], bipartite=0, node_type="respondent",
               primary_conn=row["primary_connection"])
    for issue in issue_cat.split(";"):
        issue = issue.strip()
```

```
        if issue and issue != "Unknown":
            B.add_node(issue, bipartite=1, node_type="issue",
                       color=issue_palette.get(issue, "#AAAAAA"))
            B.add_edge(row["resp_id"], issue)

resp_nodes = [n for n, d in B.nodes(data=True) if d.get("bipartite") == 0]
issue_nodes = [n for n, d in B.nodes(data=True) if d.get("bipartite") == 1]
print(f"Bipartite: {len(resp_nodes)} respondents, "
      f"{len(issue_nodes)} issue areas, {B.number_of_edges()} edges")
```

```
Bipartite: 68 respondents, 12 issue areas, 138 edges
```

```
pos = nx.spring_layout(B, k=1.5, iterations=80, seed=42)

fig, ax = plt.subplots(1, 1, figsize=(14, 10))

# Edges
nx.draw_networkx_edges(B, pos, alpha=0.08, edge_color="#666666",
                       width=0.5, ax=ax)

# Respondent nodes (circles)
r_colors = [conn_palette.get(B.nodes[n].get("primary_conn", "Unknown"),
            "#BBBBBB") for n in resp_nodes]
r_sizes = [B.degree(n) * 15 + 20 for n in resp_nodes]
nx.draw_networkx_nodes(B, pos, nodelist=resp_nodes, node_color=r_colors,
                       node_size=r_sizes, alpha=0.7, ax=ax,
                       edgecolors="white", linewidths=0.3)

# Issue-area nodes (squares)
i_colors = [issue_palette.get(n, "#AAAAAA") for n in issue_nodes]
i_sizes = [B.degree(n) * 8 + 200 for n in issue_nodes]
nx.draw_networkx_nodes(B, pos, nodelist=issue_nodes, node_color=i_colors,
                       node_size=i_sizes, alpha=0.9, ax=ax, node_shape="s",
                       edgecolors="#333333", linewidths=1.2)

# Issue-area labels
issue_labels = {n: n for n in issue_nodes}
nx.draw_networkx_labels(B, pos, labels=issue_labels, font_size=7,
                        font_weight="bold", font_color="#111111", ax=ax)

ax.set_title("Bipartite Network: Respondents and Disability Rights Issue Areas",
             fontsize=13, fontweight="bold", pad=15)
```

```
ax.axis("off")

# Legend
legend_elements = [mpatches.Patch(facecolor=c, label=k, alpha=0.8)
                   for k, c in list(conn_palette.items())[:7]]
legend_elements.append(Line2D([0], [0], marker="s", color="w",
                      markerfacecolor="#666666", markersize=10,
                      label="Issue Area (square)"))
ax.legend(handles=legend_elements, loc="lower left", fontsize=6,
          framealpha=0.9, title="Respondent Connection Type",
          title_fontsize=7, ncol=2)
plt.tight_layout()
plt.show()
```

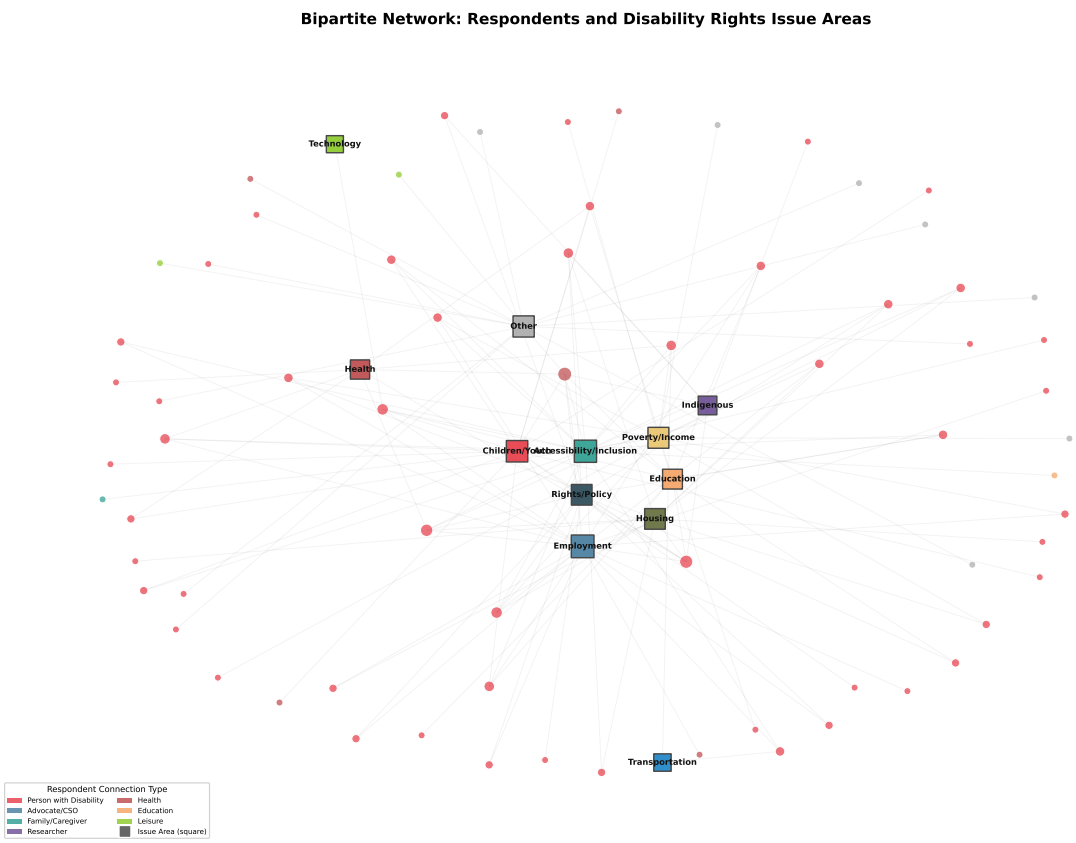

Figure 1: Figure 1. Two-mode bipartite network. Circles = respondents (coloured by connection type); squares = issue areas (sized by degree).

5 One-Mode Projected Network: Respondent Collaboration

Two respondents are linked in the projected network when they named the same organisation as a collaboration partner. Edge weight reflects the number of shared partners.

```
# Build respondent-org bipartite, then project onto respondent mode
partner_to_respondents = defaultdict(set)
for _, row in edges.iterrows():
    partner = row["partner_clean"]
    if isinstance(partner, str) and partner not in ("None", "nan"):
        partner_to_respondents[partner].add(row["respondent"])

G_proj = nx.Graph()
for _, row in edges.iterrows():
    resp = row["respondent"]
    if resp not in G_proj:
        rd = respondents[respondents["resp_id"] == resp]
        if len(rd) > 0:
            rd = rd.iloc[0]
            G_proj.add_node(resp,
                            primary_conn=rd["primary_connection"],
                            primary_issue=rd["primary_issue"],
                            language=rd["language"])

for partner, resps in partner_to_respondents.items():
    resps = list(resps)
    for i in range(len(resps)):
        for j in range(i + 1, len(resps)):
            if G_proj.has_edge(resps[i], resps[j]):
                G_proj[resps[i]][resps[j]]["weight"] += 1
            else:
                G_proj.add_edge(resps[i], resps[j], weight=1)

# Also build the full respondent-org bipartite for org centrality
G_full = nx.Graph()
for _, row in edges.iterrows():
    resp = row["respondent"]
    partner = row["partner_clean"]
    if isinstance(partner, str) and partner not in ("None", "nan"):
        G_full.add_node(resp, bipartite=0)
        G_full.add_node(partner, bipartite=1)
        fw = {"Regularly": 3, "Occasionally": 2, "Seldom": 1, "Unknown": 1}
        G_full.add_edge(resp, partner, weight=fw.get(row["freq_clean"], 1))
```

```
org_nodes_full = [n for n, d in G_full.nodes(data=True)
                  if d.get("bipartite") == 1]

print(f"Projected network: {G_proj.number_of_nodes()} nodes, "
      f"{G_proj.number_of_edges()} edges, "
      f"{nx.number_connected_components(G_proj)} components")
```

```
Projected network: 57 nodes, 17 edges, 46 components
```

```
pos_proj = nx.spring_layout(G_proj, k=2.0, iterations=100,
                            seed=42, weight="weight")

edge_weights = [G_proj[u][v]["weight"] for u, v in G_proj.edges()]
max_w = max(edge_weights) if edge_weights else 1

fig, ax = plt.subplots(1, 1, figsize=(14, 10))

nx.draw_networkx_edges(G_proj, pos_proj, alpha=0.3,
                       width=[0.5 + (w / max_w) * 3 for w in edge_weights],
                       edge_color="#999999", ax=ax)

node_colors = [conn_palette.get(G_proj.nodes[n].get("primary_conn",
               "Unknown"), "#BBBBBB") for n in G_proj.nodes()]
node_sizes = [G_proj.degree(n) * 40 + 60 for n in G_proj.nodes()]

nx.draw_networkx_nodes(G_proj, pos_proj, node_color=node_colors,
                       node_size=node_sizes, alpha=0.8, ax=ax,
                       edgecolors="#444444", linewidths=0.5)

deg = dict(G_proj.degree())
top_nodes = sorted(deg, key=deg.get, reverse=True)[:10]
labels = {n: n for n in top_nodes}
nx.draw_networkx_labels(G_proj, pos_proj, labels=labels,
                        font_size=6, font_color="#222222", ax=ax)

ax.set_title("Projected Network: Respondents Linked by Shared "
             "Collaboration Partners",
             fontsize=13, fontweight="bold", pad=15)
ax.axis("off")

legend_elements = [mpatches.Patch(facecolor=c, label=k, alpha=0.8)
                   for k, c in list(conn_palette.items())[:7]]
```

```
ax.legend(handles=legend_elements, loc="lower left", fontsize=6,
          framealpha=0.9, title="Connection Type", title_fontsize=7,
          ncol=2)
plt.tight_layout()
plt.show()
```

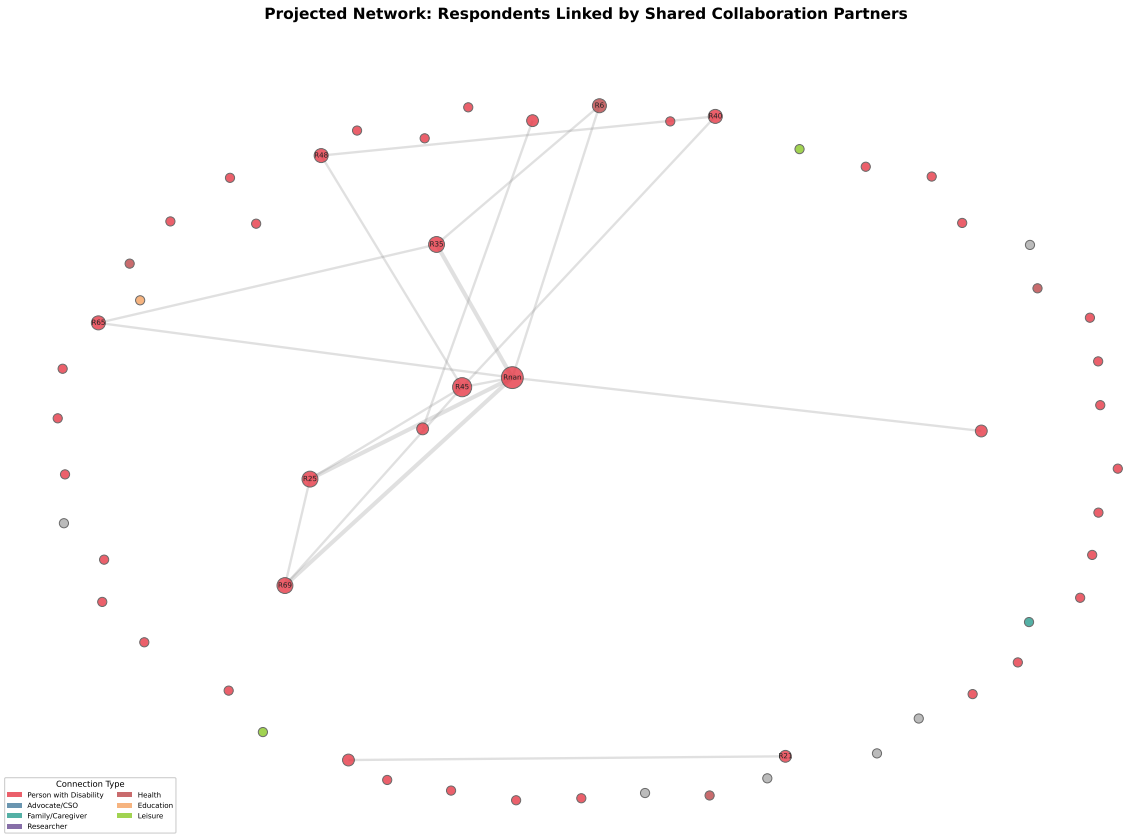

Figure 2: Figure 2. One-mode projected network. Node size = degree; colour = connection type; edge width = number of shared partners.

5.1 Network Structure Metrics

```
largest_cc = max(nx.connected_components(G_proj), key=len) \
    if G_proj.number_of_edges() > 0 else set()

metrics = [
    ["Nodes (respondents with ties)", G_proj.number_of_nodes()],
```

```
    ["Edges (shared-partner links)", G_proj.number_of_edges()],
    ["Density", f"{nx.density(G_proj):.4f}"],
    ["Connected components", nx.number_connected_components(G_proj)],
    ["Transitivity (global clustering)",
     f"{nx.transitivity(G_proj):.4f}"],
    ["Average clustering coefficient",
     f"{nx.average_clustering(G_proj):.4f}"],
    ["Largest component size", len(largest_cc)],
]

Markdown(tabulate(metrics, headers=["Metric", "Value"],
                  tablefmt="pipe"))
```

Metric	Value
Nodes (respondents with ties)	57
Edges (shared-partner links)	17
Density	0.0107
Connected components	46
Transitivity (global clustering)	0.4773
Average clustering coefficient	0.1282
Largest component size	10

6 Centrality Analysis

Centrality measures identify the most influential actors. **Degree centrality** counts direct connections; **betweenness centrality** identifies actors who bridge otherwise disconnected parts of the network.

```
org_degree = {n: G_full.degree(n) for n in org_nodes_full}
resp_degree = dict(G_proj.degree())
resp_betw = nx.betweenness_centrality(G_proj)

top_orgs_deg = sorted(org_degree.items(), key=lambda x: x[1],
                      reverse=True)[:12]
top_resp_deg = sorted(resp_degree.items(), key=lambda x: x[1],
                      reverse=True)[:12]
top_betw = sorted(resp_betw.items(), key=lambda x: x[1],
                  reverse=True)[:12]
```

```
fig, axes = plt.subplots(1, 3, figsize=(18, 7))

# Org degree
orgs_names = [x[0][:35] for x in top_orgs_deg]
orgs_vals = [x[1] for x in top_orgs_deg]
axes[0].barh(range(len(orgs_names)), orgs_vals, color="#457B9D",
             alpha=0.85, edgecolor="white")
axes[0].set_yticks(range(len(orgs_names)))
axes[0].set_yticklabels(orgs_names, fontsize=7)
axes[0].invert_yaxis()
axes[0].set_xlabel("Degree (respondents naming this org)")
axes[0].set_title("Organization Degree Centrality",
                  fontweight="bold", fontsize=10)

# Respondent degree
r_names = [f"{x[0]} ({G_proj.nodes[x[0]].get('primary_conn', '?')[:12]})"
           for x in top_resp_deg]
r_vals = [x[1] for x in top_resp_deg]
r_cols = [conn_palette.get(G_proj.nodes[x[0]].get("primary_conn",
          "Unknown"), "#BBBBBB") for x in top_resp_deg]
axes[1].barh(range(len(r_names)), r_vals, color=r_cols, alpha=0.85,
             edgecolor="white")
axes[1].set_yticks(range(len(r_names)))
axes[1].set_yticklabels(r_names, fontsize=7)
axes[1].invert_yaxis()
axes[1].set_xlabel("Degree (projected network)")
axes[1].set_title("Respondent Degree Centrality",
                  fontweight="bold", fontsize=10)

# Respondent betweenness
b_names = [f"{x[0]} ({G_proj.nodes[x[0]].get('primary_conn', '?')[:12]})"
           for x in top_betw]
b_vals = [x[1] for x in top_betw]
b_cols = [conn_palette.get(G_proj.nodes[x[0]].get("primary_conn",
          "Unknown"), "#BBBBBB") for x in top_betw]
axes[2].barh(range(len(b_names)), b_vals, color=b_cols, alpha=0.85,
             edgecolor="white")
axes[2].set_yticks(range(len(b_names)))
axes[2].set_yticklabels(b_names, fontsize=7)
axes[2].invert_yaxis()
axes[2].set_xlabel("Betweenness Centrality")
axes[2].set_title("Respondent Betweenness Centrality",
                  fontweight="bold", fontsize=10)
```

```
plt.suptitle("Centrality Analysis: Key Actors in the Canadian "
             "Disability Rights Network",
             fontsize=14, fontweight="bold", y=1.02)
plt.tight_layout()
plt.show()
```

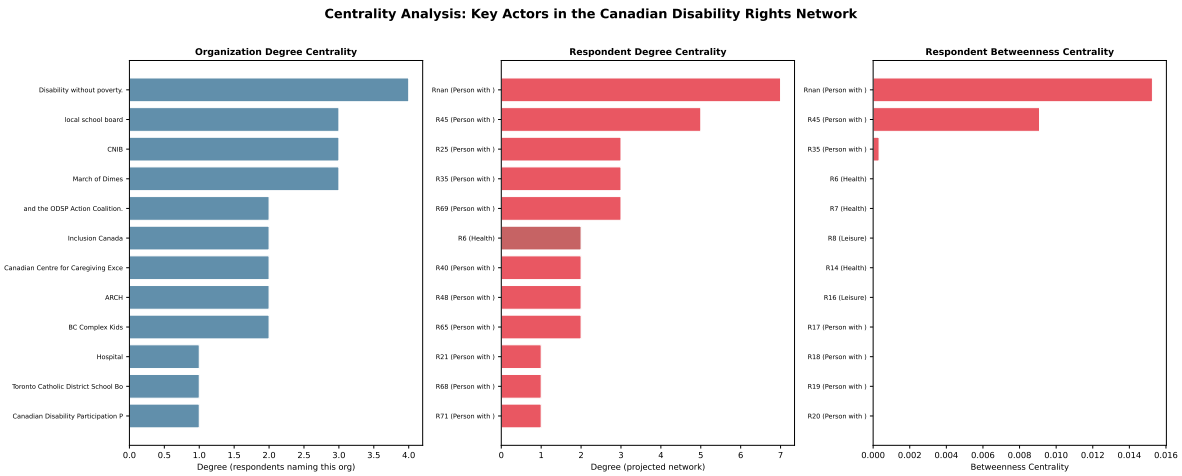

Figure 3: Figure 3. Centrality analysis. Left: Organisation degree. Centre: Respondent degree. Right: Respondent betweenness.

7 Community Detection

The Louvain algorithm partitions the network into communities that maximise modularity — groups of respondents who are more densely connected to each other than to the rest of the network.

```
partition = community_louvain.best_partition(G_proj, weight="weight",
                                             random_state=42)
n_communities = len(set(partition.values()))
modularity = community_louvain.modularity(partition, G_proj, weight="weight")
comm_sizes = Counter(partition.values())

print(f"Communities detected: {n_communities}")
print(f"Modularity: {modularity:.4f}")
print(f"Sizes: {sorted(comm_sizes.values(), reverse=True)}")
```

```
Communities detected: 47
```

```
Modularity: 0.3387
Sizes: [7, 3, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
```

```
comm_palette = ["#E63946", "#457B9D", "#2A9D8F", "#E9C46A", "#6A4C93",
                "#F4A261", "#264653", "#BC4749", "#8AC926", "#606C38",
                "#1982C4", "#FF6B6B", "#4ECDC4", "#45B7D1"]

comm_colors = [comm_palette[partition[n] % len(comm_palette)]
               for n in G_proj.nodes()]

fig, ax = plt.subplots(1, 1, figsize=(14, 10))

nx.draw_networkx_edges(G_proj, pos_proj, alpha=0.15,
                       width=[0.3 + (w / max_w) * 2 for w in edge_weights],
                       edge_color="#BBBBBB", ax=ax)
nx.draw_networkx_nodes(G_proj, pos_proj, node_color=comm_colors,
                       node_size=node_sizes, alpha=0.85, ax=ax,
                       edgecolors="#333333", linewidths=0.5)
nx.draw_networkx_labels(G_proj, pos_proj, labels=labels, font_size=6,
                        font_color="#111111", ax=ax)

ax.set_title(f"Community Detection (Louvain): {n_communities} Communities "
             f"Identified\nModularity = {modularity:.3f}",
             fontsize=13, fontweight="bold", pad=15)
ax.axis("off")

# Simplified legend: only multi-member communities
unique_comms = sorted(set(partition.values()))
legend_elements = [
    mpatches.Patch(
        facecolor=comm_palette[c % len(comm_palette)],
        label=f"Community {c+1} (n={comm_sizes[c]})", alpha=0.85
    )
    for c in unique_comms if comm_sizes[c] >= 2
]
legend_elements.append(mpatches.Patch(facecolor="#CCCCCC",
                       label="Singletons (n=1)", alpha=0.6))
ax.legend(handles=legend_elements, loc="lower left", fontsize=6,
          framealpha=0.9, title="Communities", title_fontsize=7, ncol=2)
plt.tight_layout()
plt.show()
```

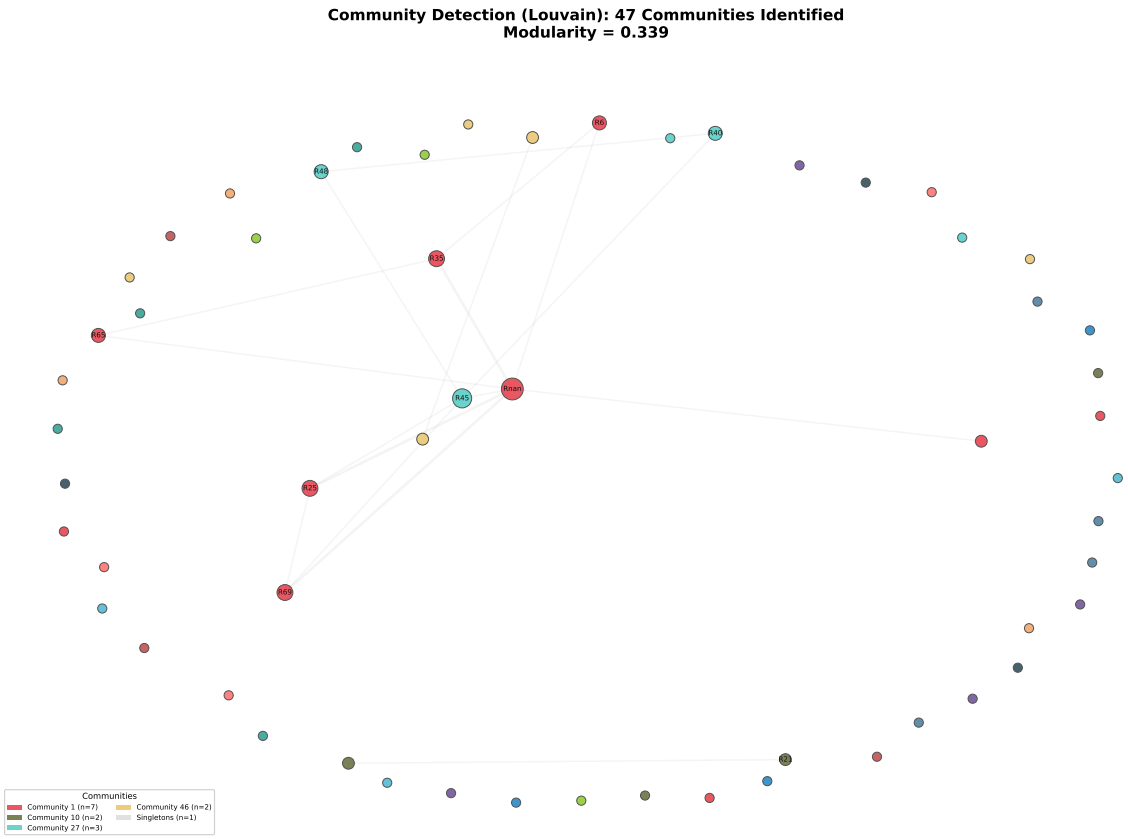

Figure 4: Figure 4. Louvain communities. Colour = community membership; size = degree.

7.1 Community Composition

```
comm_comp = {}
for node, comm in partition.items():
    conn = G_proj.nodes[node].get("primary_conn", "Unknown")
    issue = G_proj.nodes[node].get("primary_issue", "Unknown")
    if comm not in comm_comp:
        comm_comp[comm] = {"connections": [], "issues": []}
    comm_comp[comm]["connections"].append(conn)
    comm_comp[comm]["issues"].append(issue)

rows = []
for comm in sorted(comm_comp):
    if comm_sizes[comm] < 2:
        continue
```

```
    conns = Counter(comm_comp[comm]["connections"]).most_common(3)
    issues = Counter(comm_comp[comm]["issues"]).most_common(3)
    rows.append([
        f"Community {comm + 1}",
        comm_sizes[comm],
        ", ".join(f"{c[0]} ({c[1]})" for c in conns),
        ", ".join(f"{c[0]} ({c[1]})" for c in issues),
    ])

Markdown(tabulate(rows,
                  headers=["Community", "Size", "Connection Types",
                           "Issue Areas"],
                  tablefmt="pipe"))
```

Community	Size	Connection Types	Issue Areas
Community 1	7	Person with Disability (6), Health (1)	Children/Youth (3), Employment (1), Rights/Policy (1)
Community 10	2	Person with Disability (2)	Poverty/Income (1), Employment (1)
Community 27	3	Person with Disability (3)	Housing (1), Employment (1), Other (1)
Community 46	2	Person with Disability (2)	Children/Youth (2)

8 Degree Distributions and Collaboration Patterns

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Organisation degree distribution
org_degs = [G_full.degree(n) for n in org_nodes_full]
org_mean = np.mean(org_degs)
axes[0].hist(org_degs, bins=range(1, max(org_degs) + 2),
             color="#457B9D", alpha=0.8, edgecolor="white")
axes[0].axvline(org_mean, color="#E63946", linestyle="--",
                label=f"Mean={org_mean:.1f}")
axes[0].set_xlabel("Degree")
axes[0].set_ylabel("Count")
axes[0].set_title("Organization Degree Distribution",
```

```
                   fontweight="bold", fontsize=10)
axes[0].legend(fontsize=8)

# Respondent degree distribution (projected)
resp_degs = [G_proj.degree(n) for n in G_proj.nodes()]
resp_mean = np.mean(resp_degs)
axes[1].hist(resp_degs, bins=range(0, max(resp_degs) + 2),
             color="#2A9D8F", alpha=0.8, edgecolor="white")
axes[1].axvline(resp_mean, color="#E63946", linestyle="--",
                label=f"Mean={resp_mean:.1f}")
axes[1].set_xlabel("Degree")
axes[1].set_ylabel("Count")
axes[1].set_title("Respondent Degree Distribution\n(Projected Network)",
                   fontweight="bold", fontsize=10)
axes[1].legend(fontsize=8)

# Collaboration frequency (including Unknown)
freq_counts = edges["freq_clean"].value_counts()
bars = axes[2].bar(freq_counts.index, freq_counts.values,
                   color=[freq_palette.get(x, "#BBBBBB")
                          for x in freq_counts.index],
                   alpha=0.85, edgecolor="white")
axes[2].set_ylabel("Count")
axes[2].set_title("Collaboration Frequency Distribution",
                   fontweight="bold", fontsize=10)
for bar, val in zip(bars, freq_counts.values):
    axes[2].text(bar.get_x() + bar.get_width() / 2,
                 bar.get_height() + 1, str(val),
                 ha="center", fontsize=9, fontweight="bold")

plt.suptitle("Network Structure: Degree Distributions and "
             "Collaboration Patterns",
             fontsize=14, fontweight="bold", y=1.02)
plt.tight_layout()
plt.show()
```

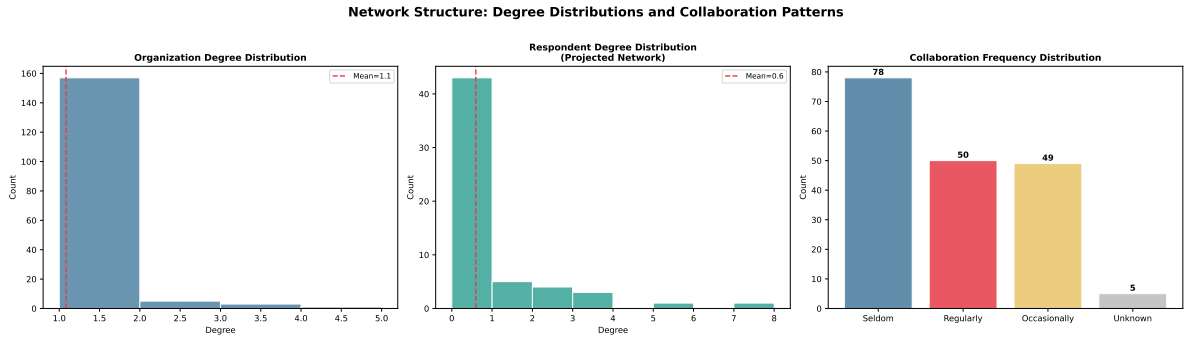

Figure 5: Figure 5. Left: Organisation degree distribution. Centre: Respondent degree distribution (projected). Right: Collaboration frequency.

9 Discussion and Implications

9.1 Fragmentation as a structural feature

The Canadian disability rights community, as captured by this survey, operates as a **highly decentralized network**. Collaboration is predominantly local and issue-specific rather than nationally integrated. This is not necessarily a weakness — it may reflect the diversity of disability issues and the importance of place-based advocacy — but it does mean that information, resources, and strategic coordination may not flow easily across the network. A small number of national organizations serve as critical connectors whose capacity to convene and bridge across issue silos is disproportionately important to overall network cohesion.

9.2 The role of national organisations

A small number of national organisations (Disability Without Poverty, CNIB, March of Dimes, Inclusion Canada) serve as critical connectors. These organisations' capacity to convene, share information, and bridge across issue silos is disproportionately important to overall network cohesion. Strengthening these hub organisations — or creating new ones — could substantially improve coordination across the community.

9.3 Bridging actors

Respondents with high betweenness centrality are structurally important: they connect otherwise isolated parts of the network. Supporting and amplifying these bridging actors — many of whom are persons with disabilities themselves — could enhance cross-pollination of ideas and strategies.

9.4 Limitations

This analysis is based on a convenience sample of 110 respondents. The free-text partner naming format means some organisations may be inconsistently recorded. The projected one-mode network relies on the assumption that shared organisational partnerships indicate a meaningful indirect connection between respondents. Finally, the survey captures reported collaboration, which may differ from actual interaction patterns.

Report generated from the Enquête d'analyse des Réseaux Sociaux / Social Network Analysis Survey. Analysis performed in Python using NetworkX, python-louvain, and matplotlib. All respondent identifiers are de-identified survey IDs.